Лабораторная Работа №3

Нурмагамбетов Данил Серикович 6203-010302D

Задание на Лабораторную Работу: Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Задание 1. Изучение стандартных классов исключений Java

Во время выполнения лабораторной работы я изучал следующие классы исключений из java.lang:

- **Exception** — базовый класс для всех проверяемых (*checked*) исключений. Используется как предок для пользовательских исключений, которые должны обрабатываться явно.
- **IndexOutOfBoundsException** — исключение времени выполнения (*unchecked*), возникающее при попытке доступа к элементу по индексу, выходящему за допустимые границы.
- **ArrayIndexOutOfBoundsException** — подкласс IndexOutOfBoundsException, специализированный для работы с массивами.
- **IllegalArgumentException** — выбрасывается, когда методу передан недопустимый аргумент (например, отрицательное количество точек или некорректные границы области определения).
- **IllegalStateException** — сигнализирует о том, что объект находится в состоянии, не позволяющем выполнить запрашиваемую операцию (например, попытка удалить точку, если их осталось менее трёх).

Эти классы позже использовались для проектирования собственной системы обработки ошибок в табулированных функциях.

Задание 2. Создание пользовательских классов исключений

В functions я создал два новых класса для работы с ошибками:

FunctionPointIndexOutOfBoundsException и **InappropriateFunctionPointException**

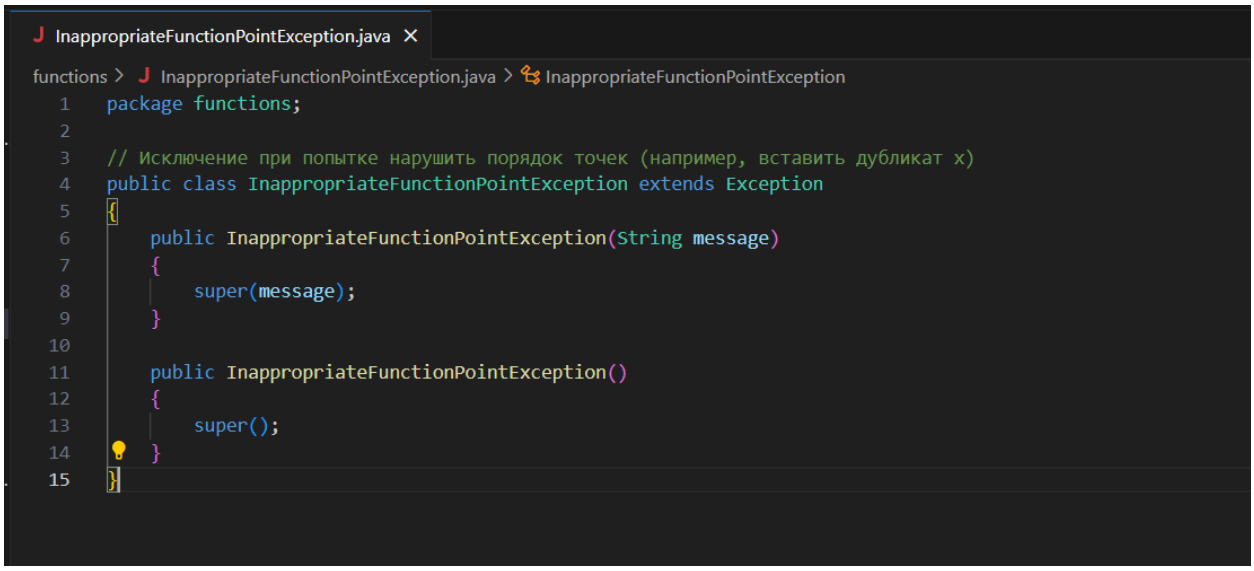


Рис.1 Реализация InappropriateFunctionPointException

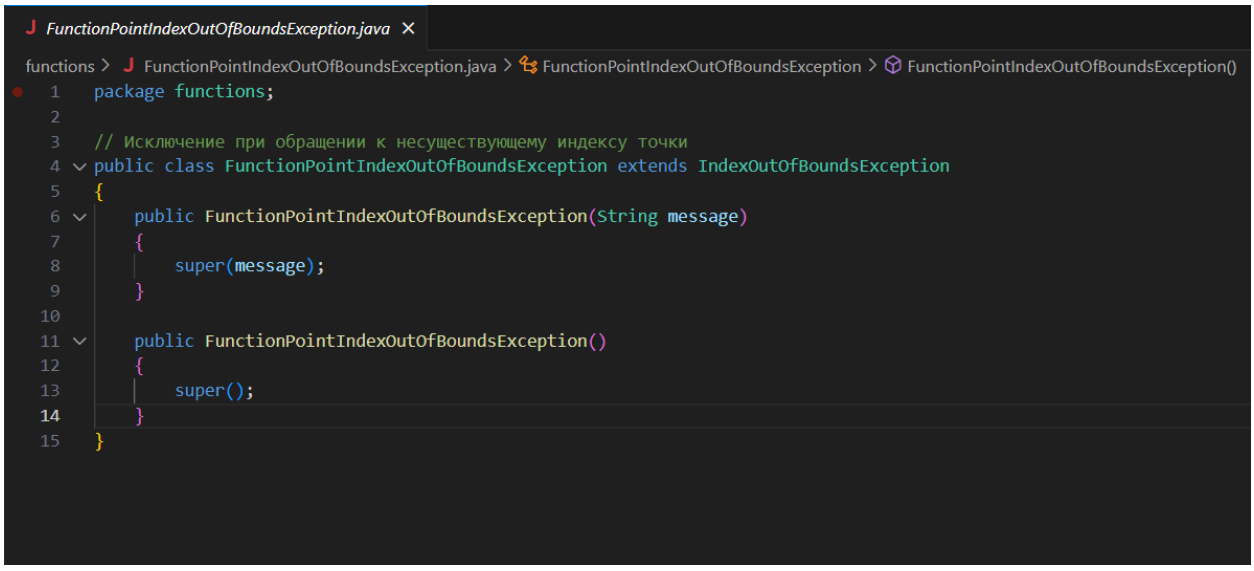


Рис.2 Реализация FunctionPointIndexOutOfBoundsException

Оба класса содержат конструкторы по умолчанию и с пользовательским сообщением об ошибке, что упрощает диагностику.

Задание 3. Модификация класса ArrayTabulatedFunction

По заданию TabulatedFunction надо было переименовать на **ArrayTabulatedFunction**, и в него нужно было внедрить систему проверок и генерации исключений, которые мы расписали в задании выше:

Конструкторы выбрасывают IllegalArgumentException, если левая граница области определения ≥ правой или (и) если количество точек < 2.

```
// Конструктор: равномерное разбиение отрезка [leftX, rightX] на pointCount точек
public ArrayTabulatedFunction(double leftX, double rightX, int pointCount)
{
    if (Double.compare(leftX, rightX) >= 0 || pointCount < 2)
    {
        throw new IllegalArgumentException(s:"Invalid parameters: leftX >= rightX or pointCount < 2");
    }

    size = pointCount;
    points = new FunctionPoint[pointCount];
    double step = (rightX - leftX) / (pointCount - 1);

    for (int i = 0; i < pointCount; i++)
    {
        points[i] = new FunctionPoint(leftX + step * i, y:0.0);
    }
}
```

```
// Конструктор: задаёт y-значения явно через массив
public ArrayTabulatedFunction(double leftX, double rightX, double[] values)
{
    if (Double.compare(leftX, rightX) >= 0 || values.length < 2)
    {
        throw new IllegalArgumentException(s:"Invalid parameters: leftX >= rightX or values.length < 2");
    }

    size = values.length;
    points = new FunctionPoint[size];
    double step = (rightX - leftX) / (size - 1);

    for (int i = 0; i < size; i++)
    {
        points[i] = new FunctionPoint(leftX + step * i, values[i]);
    }
}
```

Рис.3 Новая Реализация Конструкторов.

Методы доступа по индексу (getPoint, deletePoint и др.) проверяют корректность индекса и при нарушении выбрасывают FunctionPointIndexOutOfBoundsException.

```java
@Override
public FunctionPoint getPoint(int index) throws FunctionPointIndexOutOfBoundsException
{
    if (!isValidIndex(index))
    {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }
    return new FunctionPoint(points[index]);
}
```

Рис.4 Реализация GetPoint

```java
@Override
public void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException
{
    if (!isValidIndex(index))
    {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }
    if (size < 3) {
        throw new IllegalStateException(s:"Cannot delete: fewer than 3 points");
    }
    // Сдвигаем элементы влево, чтобы «закрыть дыру»
    System.arraycopy(points, index + 1, points, index, size - index - 1);
    points[--size] = null;
}
```

Рис.5 Реализация DeletePoint.

Методы setPoint и setPointX гарантируют, что новое значение x находится между соседними точками. В случае ошибки выбрасывается InappropriateFunctionPointException.

```java
@Override
public void setPoint(int index, FunctionPoint point)
    throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException
    {
    if (!isValidIndex(index))
        {
            throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);

        }

        double x = point.getX();

        // Определяем границы допустимого x для этой позиции

        double leftBound = (index == 0) ? Double.NEGATIVE_INFINITY : points[index - 1].getX();

        double rightBound = (index == size - 1) ? Double.POSITIVE_INFINITY : points[index + 1].getX();

        // x должен строго лежать между соседними точками

        if (Double.compare(x, leftBound) > 0 && Double.compare(x, rightBound) < 0)
        {
            points[index] = new FunctionPoint(point);
        }
        else
        {
        throw new InappropriateFunctionPointException(message:"x must be strictly between neighbors");
        }
    }
```

```java
@Override
public void setPointX(int index, double x)

throws FunctionPointIndexOutOfBoundsException, InappropriateFunctionPointException
{
    if (!isValidIndex(index))
    {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }

    // Повторно используем логику setPoint
    setPoint(index, new FunctionPoint(x, points[index].getY()));
}
```

Рис 6. Реализация SetPoint и SetPointX (SetPointY похож в реализации.)

Метод addPoint проверяет, не существует ли уже точки с таким же значением x, и при обнаружении такого совпадения, вызывается InappropriateFunctionPointException.

```java
@Override
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException
{
    double x = point.getX();
    if (isXExists(x))
    {
        throw new InappropriateFunctionPointException("Point with x=" + x + " already exists");
    }
    ensureCapacity();
    int pos = findInsertIndex(x);
    // Сдвигаем элементы вправо, чтобы освободить место
    System.arraycopy(points, pos, points, pos + 1, size - pos);
    points[pos] = new FunctionPoint(point);
    size++;
}
```

Рис.7 Реализация AddPoint.

Метод deletePoint не позволяет удалять точки, если их останется менее трёх выбросит IllegalStateException.

```java
@Override
public void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException
{
    if (!isValidIndex(index))
    {
        throw new FunctionPointIndexOutOfBoundsException("Index out of bounds: " + index);
    }
    if (size < 3) {
        throw new IllegalStateException(s:"Cannot delete: fewer than 3 points");
    }
    // Сдвигаем элементы влево, чтобы «закрыть дыру»
    System.arraycopy(points, index + 1, points, index, size - index - 1);
    points[--size] = null;
}
```

Рис.8 Реализация DeletePoint.

Задание 4. Реализация класса LinkedListTabulatedFunction

По заданию я написал **LinkedListTabulatedFunction**, хранящий точки в виде **двусвязного циклического списка.**

Внутренний класс **FunctionNode** объявлен как private static и содержит ссылки prev и next, данные для типа FunctionPoint.

Голова списка (head) узел, не хранящий данных. В пустом списке она ссылается сама на себя.

Для оптимизации доступа к элементам , нужно было кэшировать последний запрошенный узел (lastAccessed) и его индекс (lastAccessedIndex).

Метод getNodeByIndex(int index) автоматически выбирает самый короткий путь: от головы, от хвоста или от последнего использованного узла.

```
// Реализация табулированной функции на основе двусвязного списка
public class LinkedListTabulatedFunction implements TabulatedFunction
{
    private static class FunctionNode
    {
        FunctionPoint data;
        FunctionNode prev, next;

        // Фиктивная голова списка (ссылается сама на себя)
        FunctionNode()
        {
            prev = next = this;
        }

        // Узел с данными
        FunctionNode(FunctionPoint data)
        {
            this.data = data;
        }

        // Вставка узла перед текущим
        void insertBefore(FunctionNode node)
        {
            node.next = this;
            node.prev = prev;
            prev.next = node;
            prev = node;
        }

        // Удаление узла из списка
        void remove()
        {
            prev.next = next;
            next.prev = prev;
            prev = next = null;
        }
    }

    private final FunctionNode head = new FunctionNode(); // фиктивная голова
    private int size = 0;
    private FunctionNode lastAccessed = head;     // кэширование последнего доступа
    private int lastAccessedIndex = 0;
```

Рис.9 Ключевое отличие LinkedListTabulatedFunction от ArrayTabulatedFunction

Задание 5. Реализация функциональности табулированной функции в LinkedListTabulatedFunction

Класс **LinkedListTabulatedFunction** должен выполнять те же задачи, что и **ArrayTabulatedFunction** с теми же сигнатурами и теми исключениями, что и

Особое внимание я уделил **оптимизации**:

getFunctionValue(double x) осуществляет поиск интервала по списку с конца к началу, что особенно полезно при частом запросе значений вблизи правой границы.

```
// Интерполяция в связном списке (поиск с конца для ускорения)
@Override
public double getFunctionValue(double x)
{
    if (size == 0 || Double.compare(x, head.next.data.getX()) < 0 || Double.compare(x, head.prev.data.getX()) > 0)
    {
        return Double.NaN;
    }

    if (Double.compare(x, head.prev.data.getX()) == 0)
    {
        return head.prev.data.getY();
    }

    for (FunctionNode cur = head.prev; cur != head; cur = cur.prev)
    {
        if (Double.compare(x, cur.prev.data.getX()) >= 0)
        {
            return interpolate(x, cur.prev.data, cur.data);
        }
    }
    return Double.NaN;
}
```

Рис.10 GetFunctionValue

addPoint(FunctionPoint point) выполняет прямой обход списка до нахождения подходящей позиции, не вызывая getNodeByIndex в цикле.

```java
@Override
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException
{
    double x = point.getX();
    if (isXExists(x))
    {
        throw new InappropriateFunctionPointException("Point with x=" + x + " already exists");
    }

    // Вставка в начало, конец или середину
    if (size == 0)
    {
        head.insertBefore(new FunctionNode(point));
    }

    else if (Double.compare(x, head.prev.data.getX()) > 0)
    {
        head.insertBefore(new FunctionNode(point));
    }
    else
    {
        for (FunctionNode cur = head.next; cur != head; cur = cur.next)
        {
            if (Double.compare(cur.data.getX(), x) > 0)
            {
                cur.insertBefore(new FunctionNode(point));
                break;
            }
        }
    }
    size++;
    lastAccessed = head; // сброс кэша после модификации
}
```

Рис.11 AddPoint

Задание 6. Создание интерфейса TabulatedFunction

По заданию, нужно было создать единый интерфейс **TabulatedFunction**, объявляющий все общие методы: получение границ области определения,вычисление значения функции,доступ и модификация точек,добавление и удаление точек.

Оба класса ArrayTabulatedFunction и LinkedListTabulatedFunction реализуют этот интерфейс, что позволяет использовать их **полиморфно**.

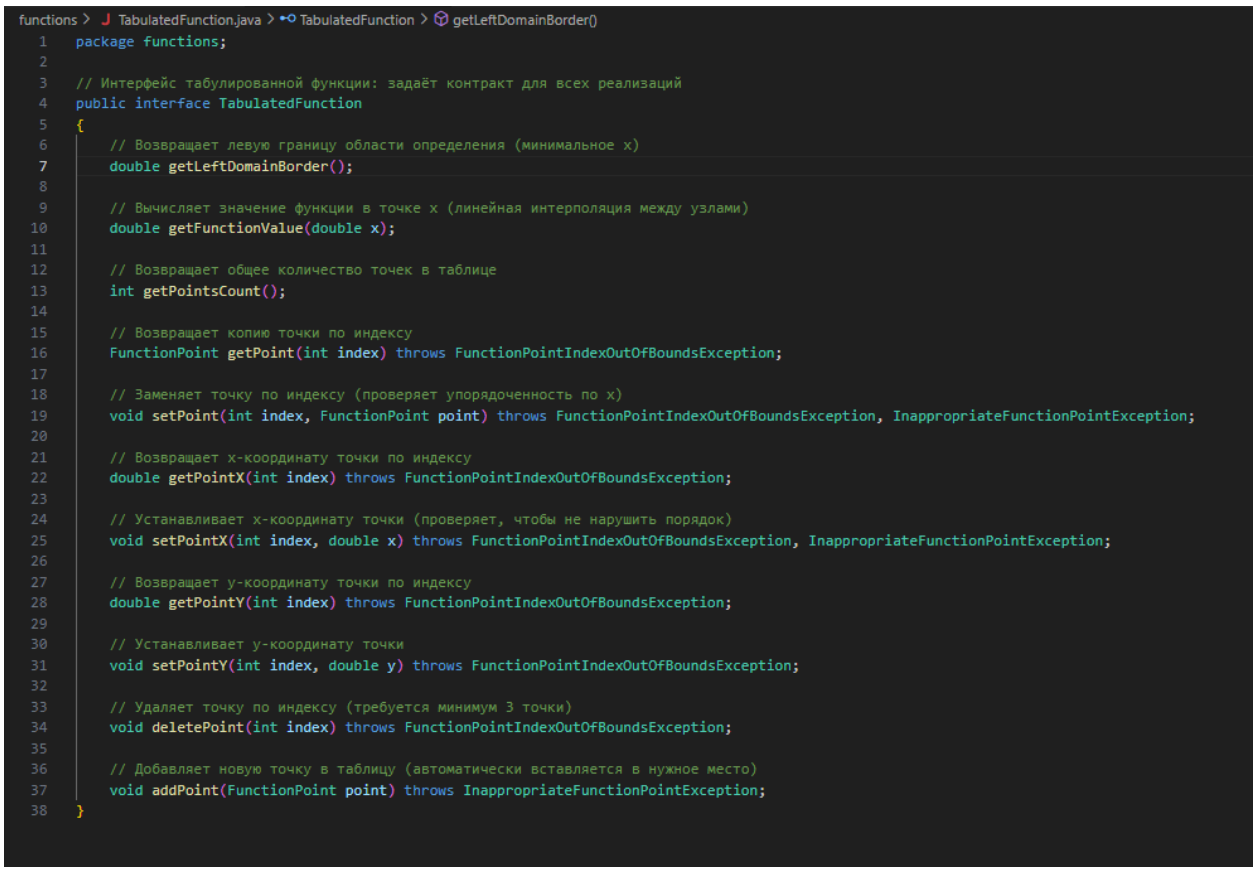


Рис.12 Полный Интерфейс TabulatedFunction

Полиморфизм это принцип, позволяющий объектам разных классов реагировать на одинаковые методы по-разному, при этом может использоваться один интерфейс

Задание 7. Проверка работы классов

В заключении, нужно было реализовать тестирование в классе Main

Тут мы тестируем – правильное создание функций (y = x^3 на отрезке [-6, 6]),добавление, удаление и модификацию точек, обработку всех типов исключений, включая:

1)выход за границы индексов,

2)нарушение порядка по x,

3)попытку удаления при количестве точек < 3,

4)некорректные параметры конструктора.

Все операции выполняются через переменную типа TabulatedFunction, что демонстрирует прозрачность использования разных реализаций.

При замене реализации с LinkedListTabulatedFunction на ArrayTabulatedFunction (и наоборот) логика программы не изменяется — подтверждая корректность архитектуры на основе интерфейса.

```
PS C:\Users\user\Desktop\Lab-3-2025> cd "c:\Users\user\Desktop\Lab-3-2025\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
Initial function:
x: NaN   y: NaN
x: -6,000        y: -216,000
x: -5,000        y: -125,000
x: -4,000        y: -64,000
x: -3,000        y: -27,000
x: -2,000        y: -8,000
x: -1,000        y: -1,000
x: 0,000         y: 0,000
x: 1,000         y: 1,000
x: 2,000         y: 8,000
x: 3,000         y: 27,000
x: 4,000         y: 64,000
x: 6,000         y: 216,000
Added (1.5, 3.375)
Deleted point at index 0
Set y=54 at index 8
Exception caught: x must be strictly between neighbors

After operations:
x: -5,000        y: -125,000
x: -4,000        y: -64,000
x: -3,000        y: -27,000
x: -2,000        y: -8,000
x: -1,000        y: -1,000
x: 0,000         y: 0,000
x: 1,000         y: 1,000
x: 1,500         y: 3,375
x: 2,000         y: 54,000
x: 3,000         y: 27,000
x: 4,000         y: 64,000
x: 5,000         y: 125,000
x: 6,000         y: 216,000
PS C:\Users\user\Desktop\Lab-3-2025>
```

Рис 13. Итог всей программы.